TECHNICAL SPECIFICATION: HUMAN-ANCHORED BYZANTINE FAULT TOLERANT NETWORK (H-BFT)

Status: PROPOSAL / CONCEPTUAL WHITE-PAPER

Paradigm: Human-Centric Network Layer Cryptography

I. INTRODUCTION & EXECUTIVE SUMMARY

The contemporary internet is structurally vulnerable to exploitation, automated Sybil attacks, and massive corporate surveillance because its baseline protocols (TCP/IP) treat digital identifiers (IP addresses, user accounts, API keys) as atomic network nodes. In the era of Generative AI, malicious actors can clone these identifiers at zero marginal cost, enabling infinite-scale automated attacks.

The **Human-Anchored Byzantine Fault Tolerant Network (H-BFT)** is a paradigm shift that fundamentally re-engineers the network layer. By anchor-linking network node authority to physical human attention, we establish a global network where the population of honest nodes (n) mathematically outweighs malicious nodes (f) under the classic Byzantine fault tolerance inequality:

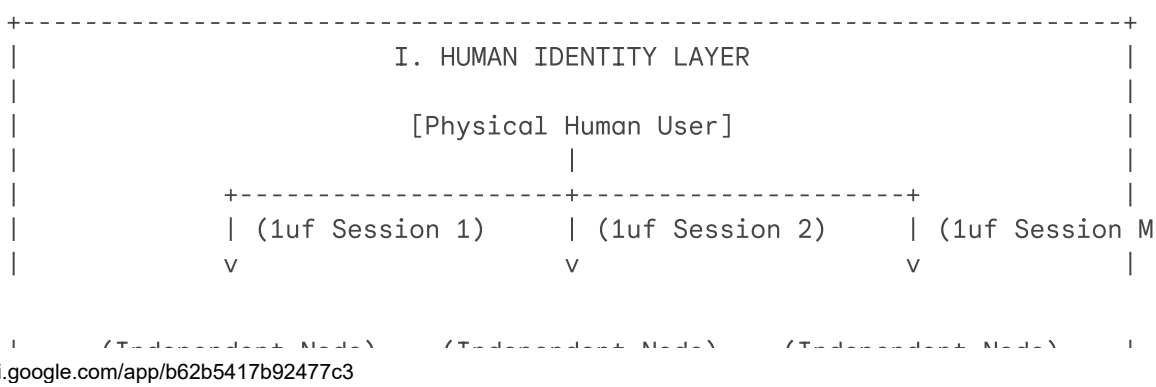
$$n \geq 3f + 1$$

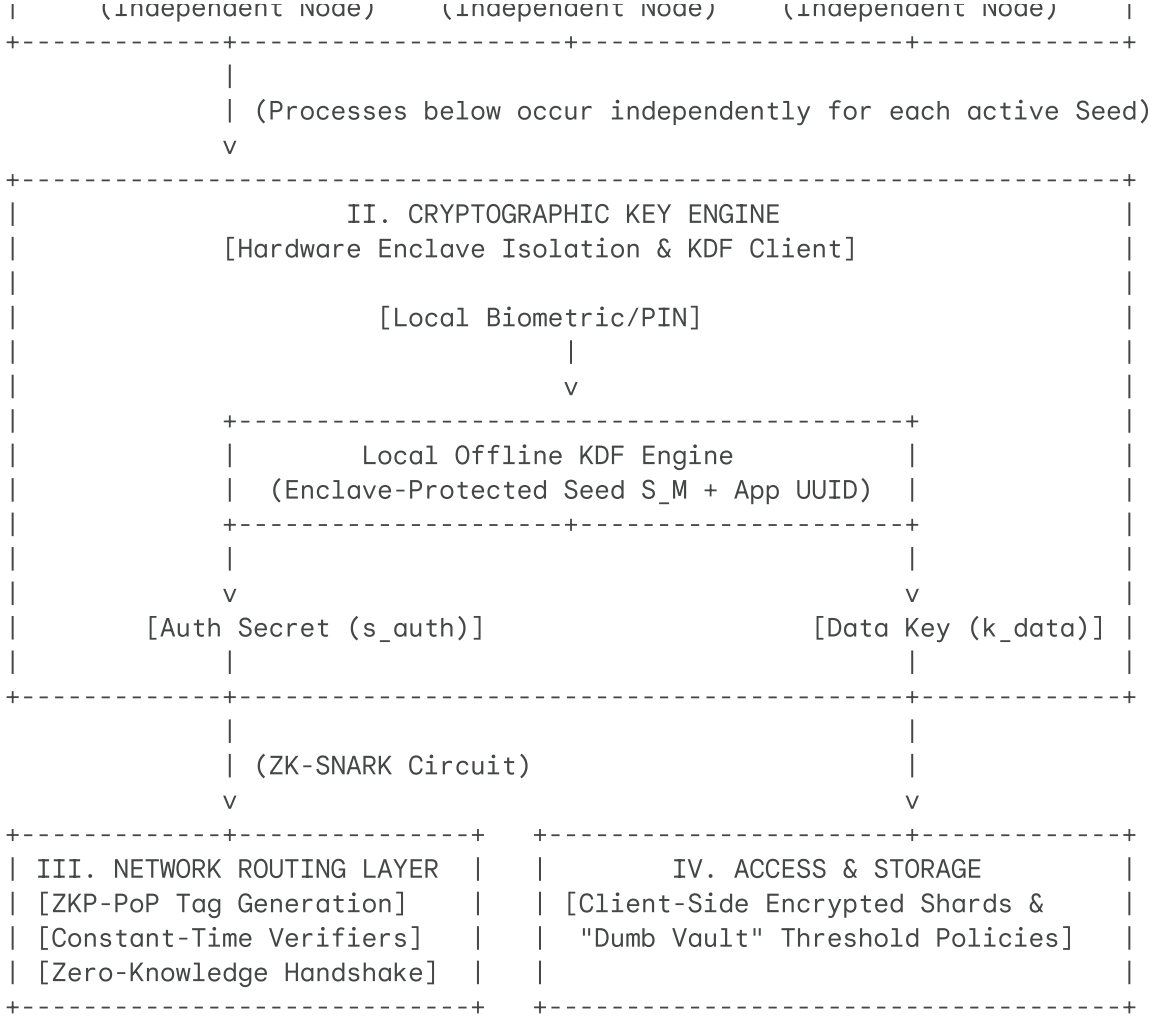
In this system, **a "Node" is defined as an active Root Seed instantiated by a single unit of human focus (1uf)**. Because human attention and time are physically finite, a single human can only spin up and maintain a small, finite number of nodes. Consequently, even a highly coordinated group of malicious actors cannot automate or "copy-paste" nodes at scale.

Through the integration of **Human Proof-of-Work**, **Zero-Knowledge Proofs (ZKPs)**, and **Threshold Cryptography**, the H-BFT protocol renders automated cybercrime, data breaches, and non-consensual tracking mathematically obsolete while preserving absolute user anonymity and allowing users to run as many distinct nodes as they choose.

II. SYSTEM ARCHITECTURE

The H-BFT architecture operates as a layered immune system, cleanly separating human verification, packet routing, and secure storage. Crucially, a single physical human can instantiate M independent, mathematically uncorrelated Root Seeds.





III. THE ATOMIC CRYPTOGRAPHIC PRIMITIVES

The H-BFT protocol is built on three tightly coupled mathematical components:

1. The 1uf (One-Unit-Focus) Engine & Attestation Leases

The 1uf serves as the physical anchor. It is a human-specific computational barrier designed to prevent automated node creation.

- **Mechanism:** A real-world interaction task that requires multi-modal human cognition (e.g., semantic analysis paired with micro-spatial interaction) executed on a local device.
- **Mathematical Property:**

$$\text{Solve}(1uf) \in \text{Human-Easy} \setminus \text{AI-Heuristic-Easy}$$

$$\text{Verify}(1uf) \in O(1)$$

are **persistent** long-term credentials. A user does not generate a new seed daily. Instead, they perform a **90-Day 1uf Re-Attestation**.

- **The Attestation Lease:** Every 90 days, the protocol requires a quick 1uf focus challenge to refresh the "Proof of Personhood" status of a persistent seed. This attestation lease guarantees that old or abandoned seeds expire automatically, preventing dead or compromised keys from indefinitely polluting the active node count (n).
- **Multi-Node Cardinality:** Because the generation and maintenance of a Root Seed requires a physical 1uf constraint, the total number of active nodes in the network is bounded by the sum of total human focus time available globally. A user can freely generate multiple seeds to partition their digital life, but an attacker cannot generate millions of fake nodes without expending millions of minutes of real human labor.

2. Domain-Separated Key Derivation (KDF) & Local Client

To maintain absolute anonymity across disparate platforms, a single Root Seed (S) is used to derive localized keys without leaving any mathematical trace that could link different accounts to the same person.

- **The Hardware Enclave Protection:** The persistent Root Seed (S) is written directly to a device's **Hardware Security Module (HSM) / Secure Enclave**. It is encrypted at rest and can never be read in plaintext by the OS or local software.
- **The Offline KDF Client:** When authenticating to a service (e.g., GitHub, Cloud Storage), the user opens an isolated, completely offline local KDF Client.
  1. The user unlocks the KDF client locally via low-latency biometrics or a local PIN.
  2. The KDF Client pings the Secure Enclave to compute the derived keys for the target domain using the seed (S) and the service's unique ID.
  3. The client outputs the derived key directly to the local application session via a secure Inter-Process Communication (IPC) pipe, ensuring the master seed is never copied to clipboard or exposed in plaintext.
- **Authentication Identifier (s_{auth}):**

$$s_{auth} = \mathrm{KDF}(S, \mathrm{Service\_UUID} \,||\, \text{"AUTH"})$$

- **Symmetric Data Key (k_{data}):**

$$k_{data} = \mathrm{KDF}(S, \mathrm{Service\_UUID} \,||\, \text{"DATA"})$$

- **Security Guarantee:** Due to the one-way properties of cryptographic hash functions, exposing s_{auth} to a service provider yields zero mathematical information about k_{data} or the underlying Root Seed (S).

3. ZKPoP (Zero-Knowledge Proof of Personhood)

exposing the seed itself.

- **The Circuit (C):** An arithmetic circuit that takes the Root Seed as a private witness (w) and proves that s_{auth} was derived correctly.
- **The Proof (π):** Generated on-device using specialized hardware accelerators.
- **The Verification:** The network validates the proof π in constant time:

$$V(\text{Public Parameters}, \pi) \rightarrow \{0, 1\}$$

IV. DECENTRALIZED DATA STORAGE & THE DIRECTORY LAYER

A critical question of this architecture is: *Where do file directories, shard mappings, and access policies live?* In a truly zero-trust system, we cannot rely on a centralized index server, as it would represent a single point of failure and metadata leakage.

1. The Decentralized Directory Ledger (DDL)

File directories, shard distribution maps, and access policies are maintained on a globally replicated, light, consensus-driven **Decentralized Directory Ledger (DDL)**. This registry is managed collectively by the active, human-verified routing and hosting nodes of the network.

An entry for a file F within the DDL contains:

$$\text{DDL\_Entry}_F = \left\{ \text{File\_ID},\ H(\text{Access\_Circuit}),\ \{\text{Encrypted\_Shard\_Map}\}_{K_{group}} \right\}$$

- **File_ID:** A unique cryptographic hash representing the file's identifier, unlinked to any human-readable name.
- **Access_Circuit Commitment:** The cryptographic hash of the access policy circuit ($H(\text{Access\_Circuit})$). This ensures the access rules (e.g., "requires a 3-of-5 threshold") are immutable. Any attempt to modify the access rules requires a broad BFT consensus update, which is rejected by the honest nodes if unauthorized.
- **Blinded Shard Map:** The addresses (IP/Routing keys) of the storage nodes hosting the physical, encrypted shards of F. This map is encrypted using the group public key (K_{group}) derived from the authorized parties' public keys.

2. Zero Leakage of Metadata

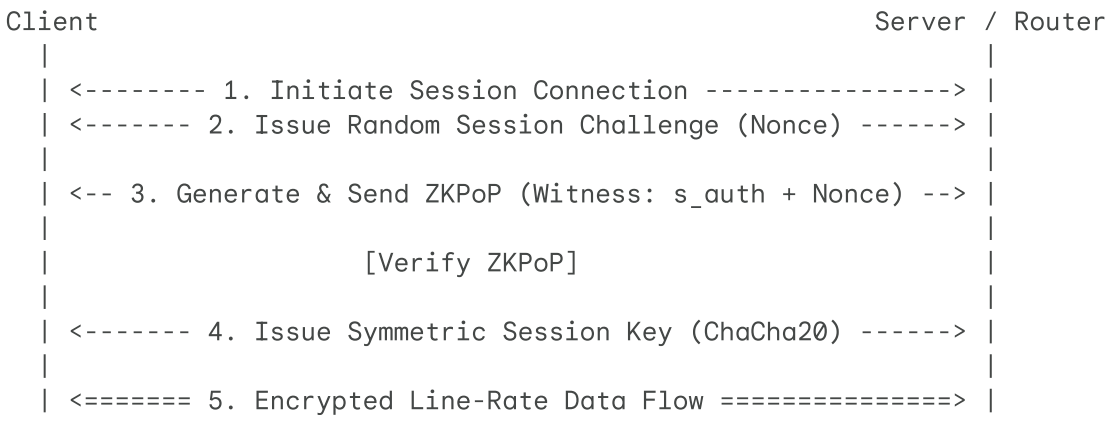
Because the shard map is encrypted under K_{group}:

- Public nodes on the DDL can see that a file with `File_ID` exists, but they **cannot** see which nodes are holding its shards, who owns the file, or what the document's file name is.
- Only a client device possessing the authorized private key can decrypt the Shard Map to locate, request, and download the raw encrypted shards from the storage nodes.

V. DATA FLOW, EXECUTIONS, & SECURE PROCESSING PIPELINES

1. The ZK-Handshake Protocol

Attaching full zk-SNARKs to every individual IP packet introduces unacceptable latency. Instead, H-BFT uses a hybrid **ZK-Handshake** model to verify human identity at the start of a session, transitioning to lightweight encryption for real-time transmission.



If a connection attempt fails to supply a valid ZKPoP during the handshake, edge routers drop the packets immediately. This eliminates automated Distributed Denial of Service (DDoS) attacks and bot network traffic at the infrastructure level.

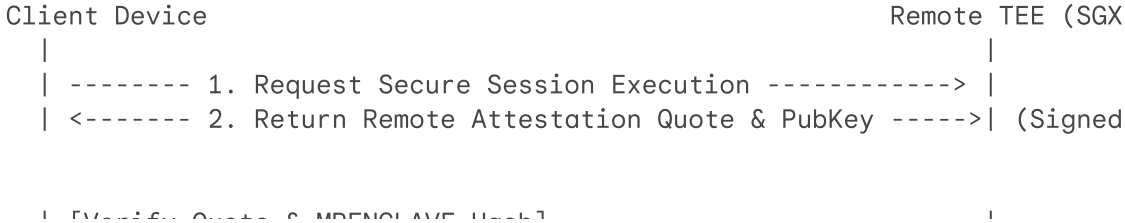
2. The "Dumb Vault" Account Architecture

Traditional apps store user profiles, plain text configurations, and telemetry data on corporate servers. Under H-BFT, corporate servers are relegated to **Dumb Vaults**:

1. **Anonymity:** The ZKPoP acts as the primary database index (the "Ghost Username"). The service provider has no name, email, or IP address linked to this index.
2. **Local Encryption:** When a user uploads files or preferences, the data is encrypted on-device using k_{data} before transmission.
3. **Storage:** The service provider stores the encrypted blob. They cannot read, monetize, or leak the data because they do not possess k_{data}.

3. Secured Outsourced Processing & Hardware-Enclave TEEs

When outsourcing intensive computation (e.g., requesting a third-party AI to summarize a highly confidential document), the data must be decrypted to be processed. This introduces the risk of network interception or host-system espionage. H-BFT solves this via **Remote Attestation** and **Silicon-Level Enclave Encryption**.



```
| [Verify Quote & MRENCLAVE Hash]                                  |
|                                                                  |
| -------- 3. Execute ECDH Key Exchange (In-Memory Only) --->|
|                                                                  |
| [Encrypt Doc using Ephemeral Key]                                |
| -------- 4. Send Ciphertext Stream over Wire ------------> |
|                                                                  |  [Decrypt to L3
|                                                                  |  [Process in Enc
|                                                                  |  [Encrypt Output
| <------- 5. Return Encrypted Result ---------------------- |
```

The Execution Mechanics:

1. **Hardware Verification (Remote Attestation):** Before Sarah's device pings any data to a remote service, the remote CPU (e.g., Intel SGX or AMD SEV) must generate an **Attestation Quote**. This quote is cryptographically signed by the CPU manufacturer's hardware root key. It proves two things to Sarah's device:
   - The destination is a genuine, untampered physical secure enclave.
   - The enclave is running the *exact, un-modified open-source code* expected (verified by matching the cryptographic hash of the memory image, known as `MRENCLAVE`).
2. **The Interception Shield (ECDH Key Exchange):** Once attestation is verified, Sarah's device and the remote CPU perform an **Ephemeral Elliptic Curve Diffie-Hellman (ECDH)** key exchange *directly into the TEE's hardware-isolated memory*. No external network listener, hypervisor, host OS, or company administrator can intercept this key exchange.
3. **Silicon-Level Memory Encryption:** Sarah's device encrypts the document with this ephemeral key and streams it. The remote processor receives the ciphertext and decrypts it *only inside the hardware-encrypted L3 cache lines of the CPU*.
4. **Host Defiance:** The hosting company trying to profit from the TEE cannot "peek" at their own hardware. The system memory (RAM) allocated to the enclave is encrypted at the silicon level by the CPU's hardware memory encryption engine. Any hardware probe, memory dump, or administrative access attempt to read the enclave memory yields only scrambled, high-entropy cryptographic noise. The data is processed in complete, hardware-guaranteed isolation, and is purged instantly upon session completion.

VI. SURVEILLANCE, TELEMETRY, & METADATA SANITIZATION

A persistent issue with modern platforms is that even if a service cannot decrypt your files, it can collect side-channel metadata (user-agent details, network latency, device specifications, typing speeds) to build a behavioral profile linked to your ZKPoP. H-BFT resolves this at both the client OS and the network transport layer.

1. Edge-Level Telemetry Sandboxing

In the H-BFT ecosystem, client operating systems run a built-in **Network Telemetry Sanitizer**:

environments. The OS spoofs and normalizes all outward-facing environment variables (e.g., standard virtual screen dimensions, generic browser headers, uniform system clock offsets).

- **Outgoing Firewalls:** If an application attempts to write metadata packets back to its parent servers without an explicit, cryptographically authenticated user-consent signature, the OS kernel drops the packet at the hardware network interface card (NIC).

2. Network Packet Anonymization Standard (NPAS)

To prevent network-level metadata analysis (such as IP correlation and traffic timing analysis), H-BFT incorporates a decentralized **Multi-Layer Mixnet**:

- **The Layered Protocol:** Standard TCP/IP is wrapped inside NPAS. Packet routing is handled via onion-style routing where every packet is routed through multiple randomized intermediate human-verified nodes (n).
- **Verification and Routing:** Because intermediate transport nodes are verified human nodes ($n \geq 3f + 1$), they are bound by the BFT consensus rules of the protocol. If an intermediate node attempts to inject tracking tags, log transit metadata, or correlate IP paths, the surrounding network nodes mathematically detect the protocol deviation and immediately drop/slash the offender.
- **The Destination:** By the time a packet arrives at the "Dumb Vault" service, the incoming IP address is that of the final mixnet exit node, completely decoupling the user's physical location and network identity from their anonymous ZKPoP account identifier.

VII. MODULAR SOVEREIGNTY & DISTRIBUTED RECOVERY

Unlike identity systems that lock a human's entire physical existence to a single master credential, the H-BFT protocol treats digital identity as modular and partitioned.

1. Multi-Seed Identity Partitioning

Because a user can spend as many 1uf sessions as they want, they can structure their digital life across multiple independent Root Seeds ($S_1, S_2, \ldots$):

- **Ephemeral/Burner Seeds:** Created for casual web browsing, test accounts, or public forums. These require no backups or recovery mechanisms. If lost or compromised, the user simply discards the seed and spins up another.
- **Sovereign/High-Value Seeds:** Created for highly sensitive storage, financial assets, or primary enterprise credentials. These are heavily protected and use recovery protocols.

2. Decentralized Social Recovery for High-Value Seeds

For high-value seeds where recovery is desired, the protocol implements **Decentralized Social Recovery** without compromising the absolute anonymity of the underlying owner:

1. **Sharding the Seed:** The user's device splits a specific high-value Root Seed (S) into y distinct pieces using Shamir's Secret Sharing.
2. **Trusted Guardians:** The user distributes these pieces to the secure enclaves of y trusted peer nodes (e.g., 5 close friends, family members, or secondary personal devices).

destroyed, the user can request their guardians to sign a recovery transaction. Once a threshold x (e.g., 3 out of 5) of the guardians approve the request, the specific Root Seed is mathematically reconstructed on the user's new device.

4. **Zero-Knowledge Isolation:** Throughout this process, no individual guardian can see the parent seed, find out which specific services are tied to that seed, or access the user's encrypted data. The recovery process is strictly a blind mathematical reconstruction.

VIII. CORPORATE ECONOMIC INCENTIVES & SYSTEM ADOPTION

For any privacy-preserving system to succeed, it must align with real-world enterprise incentives. Businesses adopt H-BFT not out of altruism, but to secure clear financial, risk-reduction, and structural advantages.

1. Complete Offloading of Data Breach Liability

Under modern regulations (GDPR, CCPA, HIPAA), companies face multi-million-dollar fines and existential brand damage if they are breached.

- By converting corporate storage into **Dumb Vaults**, companies no longer hold readable consumer data.
- If a hacker breaches a company's storage servers, they steal only scrambled, client-side encrypted shards indexed by anonymous ZKPoPs.
- Under global privacy laws, this completely neutralizes "Data Breach" disclosure requirements and associated fines, transferring all data storage risk away from the enterprise balance sheet.

2. Elimination of Ransomware Threats

Ransomware works by exfiltrating sensitive data and threatening public release. Because data is sharded and client-side encrypted at rest under H-BFT, hackers cannot read or weaponize the stolen data. The ransom incentive structure is entirely broken.

3. Radical Security Infrastructure Cost Reductions

Enterprises currently spend billions of dollars annually maintaining active threat monitoring systems, Security Operations Centers (SOCs), firewalls, and complex identity access management (IAM) softwares.

- H-BFT replaces this fragile "perimeter defense" with immutable, protocol-level mathematics.
- Companies can scale down their security engineering divisions to basic hardware hosting and network maintenance, driving massive reductions in operational expenditure (OpEx).

IX. CONSENSUS MECHANICS & PERFORMANCE OPTIMIZATION

To achieve sub-second global response times across billions of components without sacrificing cryptographic safety, H-BFT operates via a strict structural separation of processing planes.

1. Separation of Data and Control Planes

The protocol enforces a total isolation of data transit from network state coordination:

processing queries execute peer-to-peer over point-to-point lines. This traffic relies entirely on pre-shared symmetric encryption (ChaCha20-Poly1305) established via the ZK-Handshake. It incurs an algorithmic overhead of O(1). BFT consensus algorithms are completely excluded from this plane.

- **The Control Plane (Asynchronous Slow Path):** BFT consensus is isolated strictly to infrastructure orchestration on the Decentralized Directory Ledger (DDL). The ledger manages network topology configurations, state directory mapping roots, identity verification parameters, and permission blacklists. It runs concurrently and asynchronously alongside active traffic streams.

2. BFT Consensus Triggers (When Consensus Runs)

BFT consensus engines operate on an event-driven and epoch-segmented schedule rather than a per-packet basis:

- **Epoch Boundary State Commitments:** Every 24 hours (one Network Epoch), a batch consensus cycle occurs across the network. This cycle formalizes new anonymous Root Seed registrations, processes 90-day 1uf lease extensions, and garbage-collects expired or un-attested keys.
- **Directory State Changes:** Any active, user-initiated modification to the structural state of a file drive or administrative policy (such as changing a corporate M&A access policy parameter or publishing a new public contract) pushes a transaction to the DDL queue, prompting a consensus confirmation session.
- **Protocol Violation Slashing:** If an active network node attempts data corruption, timing manipulation, or metadata insertion, surrounding observer nodes instantly broadcast a zero-knowledge fraud proof. This triggers a localized BFT session to quarantine and permanently slash the rogue node's stake and lease.

3. Sub-Committee Sharding & Asynchronous DAG Engines (How Consensus Scales)

To maintain constant-time network performance, H-BFT scales its infrastructure verification workload through localized sharding and non-linear block construction:

- **VRF-Selected Sub-Committees:** The entire network size (n) does not participate in every transaction. For a given ledger or file directory shard, a Verifiable Random Function (VRF) cryptographically selects a dynamic, small validator sub-committee (e.g., 200 nodes) based on localized topological proximity and active 1uf attestation status. This collapses the traditional O(N^2) message-passing bottleneck of BFT networks.
- **Asynchronous DAG Memory Engines:** The selected sub-committees build state changes utilizing a Directed Acyclic Graph (DAG) architecture. Instead of processing synchronous,